

# Extending the Linear Model 6: Generalised Linear Models Lab

Dr. Luisa Barbanti and Dr. Matteo Tanadini

Applied Machine Learning and Predictive Modelling 1, HS25 (HSLU)

## Contents

|          |                                                                                         |           |
|----------|-----------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Count data</b>                                                                       | <b>3</b>  |
| 1.1      | Introductory example . . . . .                                                          | 3         |
| 1.2      | The Poisson Model . . . . .                                                             | 7         |
| 1.3      | The Poisson Model in $\mathbf{R}$ . . . . .                                             | 8         |
| 1.4      | Take Home Messages (count data) . . . . .                                               | 10        |
| <b>2</b> | <b>Binary and binomial data</b>                                                         | <b>12</b> |
| 2.1      | Example . . . . .                                                                       | 12        |
| 2.2      | The “bliss” data set . . . . .                                                          | 12        |
| 2.3      | Link function for binomial data . . . . .                                               | 16        |
| 2.4      | The Binomial Model in $\mathbf{R}$ . . . . .                                            | 18        |
| 2.5      | Take Home Messages (binary/binomial data) . . . . .                                     | 19        |
| <b>3</b> | <b>Further on binary and binomial data</b>                                              | <b>20</b> |
| 3.1      | Binary data: Logistic regression . . . . .                                              | 20        |
| 3.2      | Multinomial data (*) . . . . .                                                          | 20        |
| 3.3      | Estimating the performance of a binary model . . . . .                                  | 21        |
| <b>4</b> | <b>Generalised Linear Models</b>                                                        | <b>25</b> |
| <b>5</b> | <b>Appendix</b>                                                                         | <b>26</b> |
| 5.1      | Interpreting the coefficients of a Poisson Model (*) . . . . .                          | 26        |
| 5.1.1    | Interpreting the coefficients of a Poisson Model: Factors (*) . . . . .                 | 26        |
| 5.1.2    | Interpreting the coefficients of a Poisson Model: Continuous predictors (*) . . . . .   | 28        |
| 5.2      | Interpreting the coefficients of a Binomial Model (**) . . . . .                        | 29        |
| 5.2.1    | Interpreting the coefficients of a Binomial Model: Continuous predictors (**) . . . . . | 29        |
| 5.2.2    | Interpreting the coefficients of a Binomial Model: Factors (**) . . . . .               | 31        |
| 5.2.3    | The “divide by 4 rule” . . . . .                                                        | 32        |

|                            |                                            |           |
|----------------------------|--------------------------------------------|-----------|
| 5.3                        | Overdispersion (**)                        | 34        |
| 5.3.1                      | Overdispersion: In Poisson Models (**)     | 34        |
| 5.3.2                      | Overdispersion: In Binomial Models (**)    | 35        |
| 5.4                        | GAMs and GLMs                              | 35        |
| 5.5                        | What if my predictors are e.g. count data? | 36        |
| 5.6                        | Not all proportions are binomial data      | 36        |
| <b>Session Information</b> |                                            | <b>38</b> |

# 1 Count data

## 1.1 Introductory example

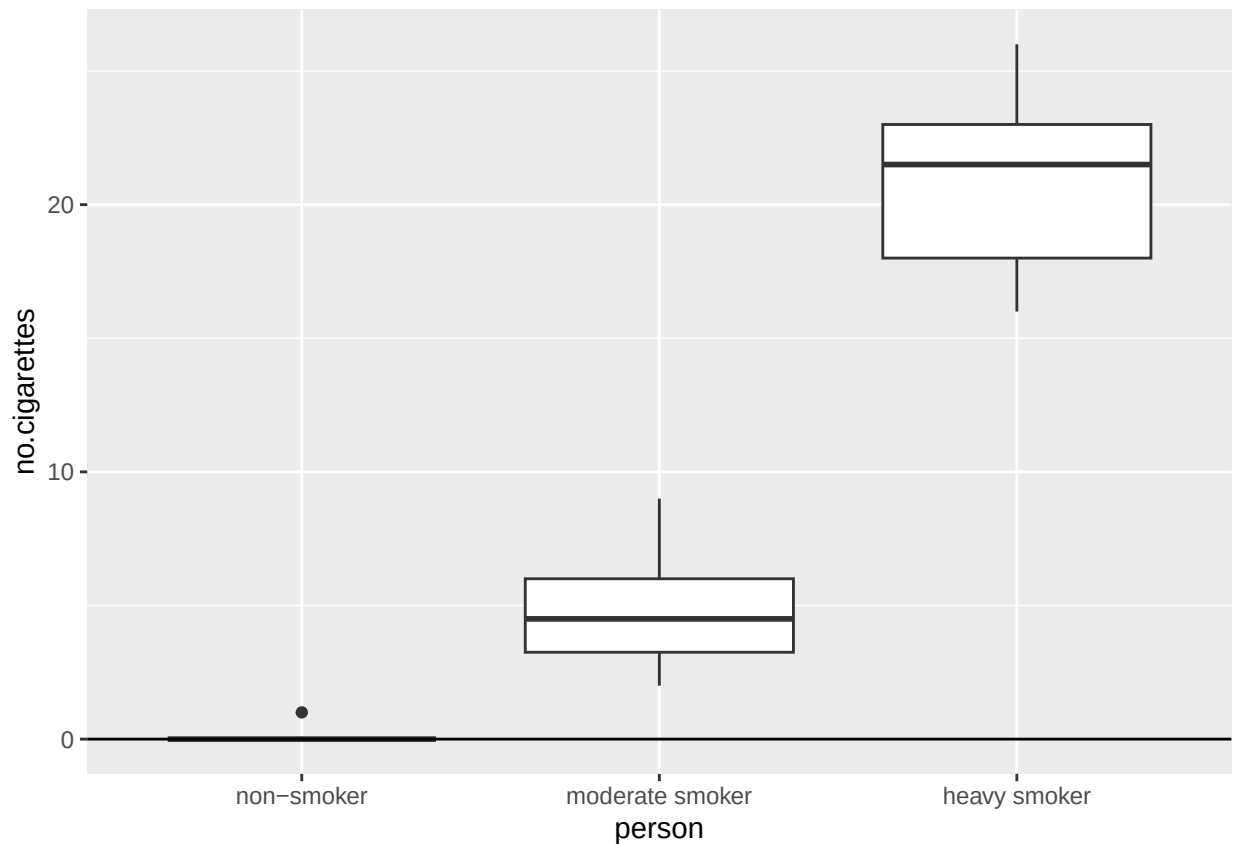
Up to this point we have dealt with the Linear Model and its extension to non-linearity (i.e. the GAM model). In these models, the response variable is a continuous variable that is assumed to follow a normal distribution. Let's now turn our attention to another type of data often encountered in real cases: Count data. Below we list a few examples of count data:

- items purchased by one client during a single shopping session
- number of children in a family
- faulty parts found in a car during an inspection
- number of cigarettes smoked daily by a given person

**Question:** Assume you have three friends i) a non-smoker, ii) someone who smokes about 5 cigarettes per day and iii) another person that smokes about a pack (i.e. 20 cigarettes) per day. Assume you are ask them how many cigarettes they smoked on each day during the past two week. Please simulate  $14 * 3$  observations that you think are likely to occur in this situation.

**Question:** Draw a boxplot (by hand) for each one of your friends using to the data you just “simulated”. What patterns do you observe?

This is a possible realisation of the situation depicted above, where we simulate two weeks worth observations for the three smokers.



### Modelling count data with a Linear Model

If we were to use a Linear Model to model these data, we would assume the data to follow a normal distribution. To test whether this would be a good way to proceed we fit a Linear Model to the data displayed in the graph above. If you prefer, you can fit a Linear Model to the data that you simulated.

In the data set I simulated (simulation is not shown), the response variable is *no.cigarettes* and the predictor is *person*. Each observation represents the number of cigarettes smoked per day. Data are available for a two weeks period (i.e.  $14 * 3 = 42$  observations in total).

```
lm.smokers <- lm(no.cigarettes ~ person, data = d.smokers)
round(coef(lm.smokers), digits = 1)
```

|             |        |                 |        |              |
|-------------|--------|-----------------|--------|--------------|
| (Intercept) | person | moderate smoker | person | heavy smoker |
| 0.1         |        | 5.0             |        | 20.8         |

The estimates of this Linear Model, which assumes normality of the data, look good. Indeed, we have zero cigarettes for the *non smoker*, about five for the *moderate smoker* and about 20 cigarettes for the last person. This is exactly what we observe on the graph above. So, for what concerns the regression coefficients the Linear Model still seems an appropriate way to analyse count data.

Let's dig a bit further and simulate some new observations from this Linear Model (note that **R** enables the user to simulate new observations from virtually any fitted model via the `simulate()` function).

```
set.seed(3)
sim.data.smokers <- simulate(lm.smokers)
##
NROW(sim.data.smokers)
```

```
[1] 42
```

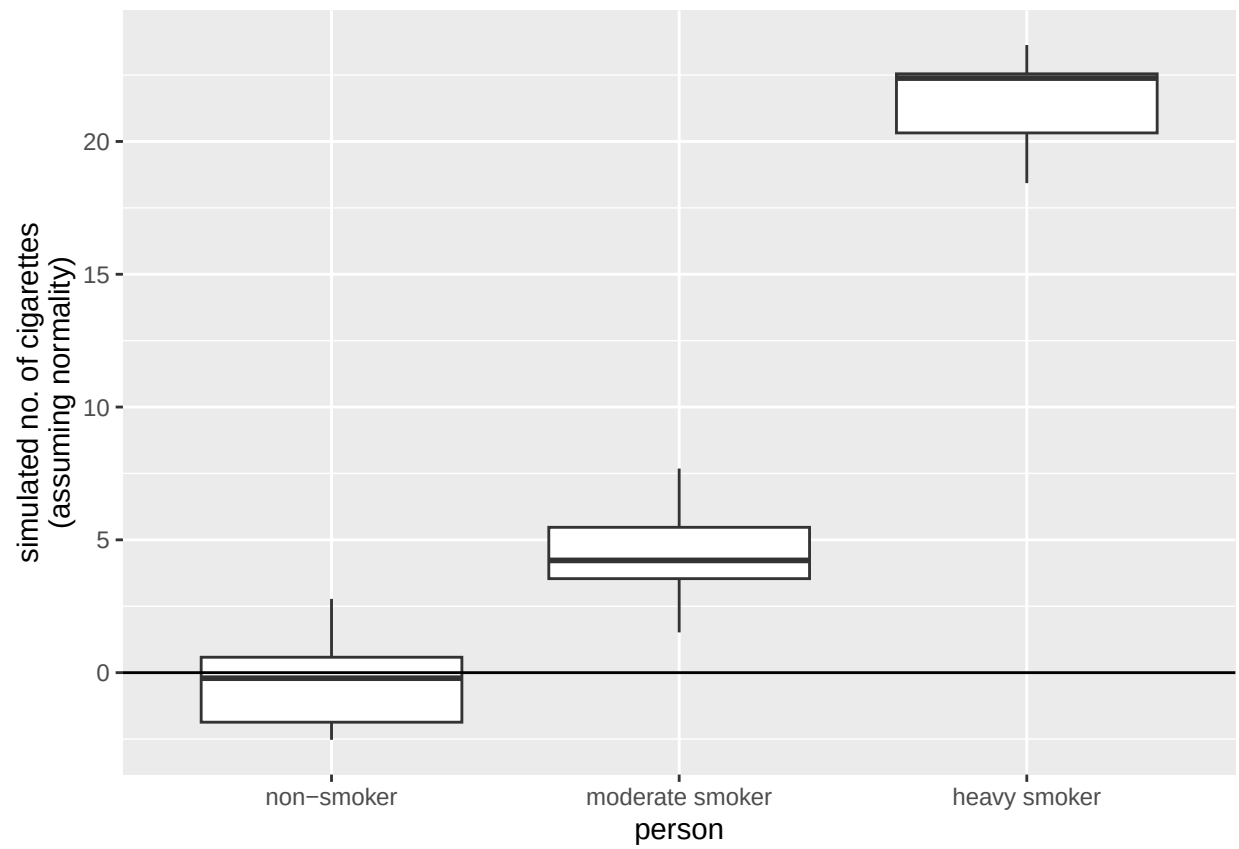
```
head(sim.data.smokers)
```

```
      sim_1
1 -1.9811
2 -0.5528
3  0.6236
4 -2.3870
5  0.4892
6  0.1357
```

```
tail(sim.data.smokers)
```

```
      sim_1
37 23.63
38 20.94
39 18.77
40 22.55
41 22.54
42 20.19
```

```
##
ggplot(mapping = aes(y = sim.data.smokers$sim_1,
                     x = d.smokers$person)) +
  geom_boxplot() +
  geom_hline(yintercept = 0) +
  ylab("simulated no. of cigarettes\n(assuming normality)") +
  xlab("person")
```



**Question:** *What patterns differ between the simulated data and the real data?*

There are some striking differences between the data we simulated from the Linear Model and the data that I previously made up:

- the data I made up are integers, the data simulated from the Linear Model are numeric (note that none of the simulated values is an integer)
- the data I made up are always larger than zero, the data simulated from the Linear Model goes below zero as well (i.e. a negative number of cigarettes...)
- the variability in the data I made up increases with the mean value of the person (i.e. “variability non-smoker” < “variability moderate-smokers” < “variability heavy-smoker”). On the other hand, the variability of the simulated values is the same in all three groups.

## 1.2 The Poisson Model

So, to reiterate, fitting a Linear Model to the count data I made up is inconvenient as:

1. simulated and fitted values can be negative (but a negative count is nonsensical in reality)
2. simulated values are not integers (count data are always integers)
3. for count data we naturally observe that the variability of the observations increases with the expected value. For example, we expect the variability for the heavy smoker to be larger than the variability observed in the moderate smoker.

Let’s see how we could modify the Linear Model such that these requirements that come from the very nature of count data are fulfilled.

### Avoiding negative fitted values (1.)

Consider the equation we have used so far when computing the fitted values of a Linear Model.

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 \cdot x_1$$

To solve the problem of negative fitted values, we must make sure that the right hand-side of this equation always produces positive results. One solution is to apply a transformation to it. For example, we can use the exponential function:

$$\hat{y} = \exp\left(\hat{\beta}_0 + \hat{\beta}_1 \cdot x_1\right)$$

In this case, the fitted values will always be positive. Indeed, regardless of the results of  $\hat{\beta}_0 + \hat{\beta}_1 \cdot x_1$  its exponential will always be a positive number. See a few examples below.

```
exp(-5)
```

```
[1] 0.006738
```

```
exp(-2)
```

```
[1] 0.1353
```

```
exp(0)
```

```
[1] 1
```

```
exp(3)
```

```
[1] 20.09
```

Note that we can rewrite the above equation by taking the logarithm on both sides. The equation then becomes:

$$\log(\hat{y}) = \hat{\beta}_0 + \hat{\beta}_1 \cdot x_1$$

### Integers and non-constant variance (2. and 3.)

Now we also need to make sure that the simulated values are integers and that variance of the observations depends on the mean values. The solution to these problems is to assume another distribution of the data. For count data, the classical solution is to assume the data to follow a Poisson distribution (rather than the normal distribution).

The Poisson Model, becomes then:

$$\log(y) = \beta_0 + \beta_1 \cdot x_1 \quad y \sim \text{Poisson}(\lambda)$$

This model fulfills all the three requirements we listed above<sup>1</sup>.

As a matter of fact, for the Poisson distribution the variance of the observations linearly increases with the mean value. For example, if we were to assume the Poisson distribution for the smokers data, the group with mean 5 (i.e. the moderate smoker) would have a variance of 5. On the other hand, the group with mean 20, would have a variance of 20.

## 1.3 The Poisson Model in R

To fit a Poisson Model in **R** we can use the `glm()` function, where we specify the distribution to be used. Note that GLM stands for Generalised Linear Model. Indeed, GLMs generalise the Linear Model in the sense that the data can follow distributions other than normal.

```
glm.smokers <- glm(no.cigarettes ~ person,
                  family = "poisson", ## we specify the distribution!
                  data = d.smokers)
```

Side note: A very common (beginners-) mistake when fitting GLMs is forgetting to set the “family” argument. Note that the default family in `glm()` is “gaussian”. Therefore, if you don’t specify the family argument... you are simply fitting a plain Linear Model.

Let’s look at the summary output.

---

<sup>1</sup>You can convince yourself that this is true, by simulating new observation from the Poisson distribution (in **R** you may want to use the `rpois()` function).

```
summary(glm.smokers)
```

Call:

```
glm(formula = no.cigarettes ~ person, family = "poisson", data = d.smokers)
```

Coefficients:

|                       | Estimate | Std. Error | z value | Pr(> z )    |
|-----------------------|----------|------------|---------|-------------|
| (Intercept)           | -2.64    | 1.00       | -2.64   | 0.0083 **   |
| personmoderate smoker | 4.26     | 1.01       | 4.23    | 2.3e-05 *** |
| personheavy smoker    | 5.68     | 1.00       | 5.67    | 1.5e-08 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 450.034 on 41 degrees of freedom  
Residual deviance: 22.847 on 39 degrees of freedom  
AIC: 146.9

Number of Fisher Scoring iterations: 6

For the moment we do not interpret the summary (see Appendix for more information). Note, however, that the structure of the summary output is similar to the one obtain when using the `lm()` function (**R** tries to be consistent across methods).

Let's simulate some data from this model.

```
set.seed(2)
sim.data.smokers.Poisson <- simulate(glm.smokers)
##
NROW(sim.data.smokers.Poisson)
```

```
[1] 42
```

```
head(sim.data.smokers.Poisson)
```

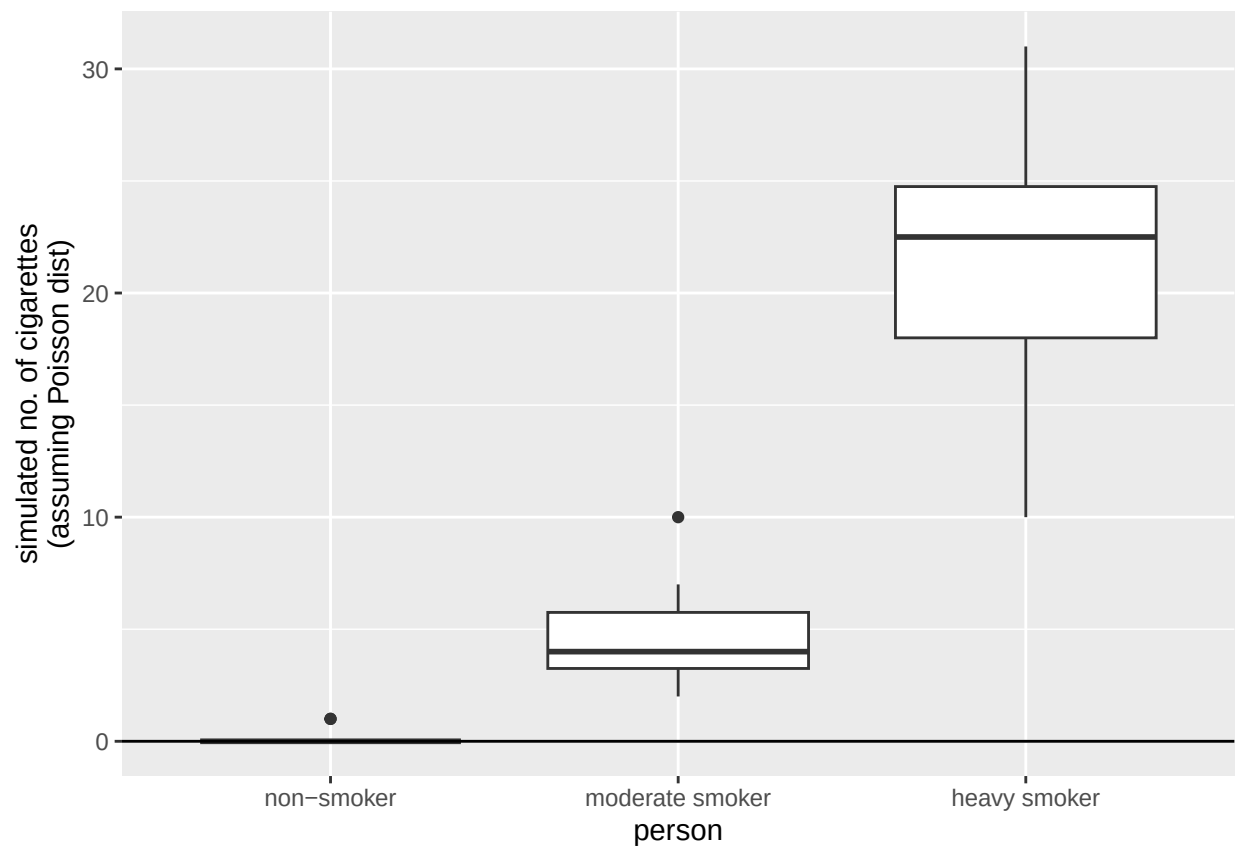
|   | sim_1 |
|---|-------|
| 1 | 0     |
| 2 | 0     |
| 3 | 0     |
| 4 | 0     |
| 5 | 1     |
| 6 | 1     |

```
tail(sim.data.smokers.Poisson)
```

|    | sim_1 |
|----|-------|
| 37 | 19    |
| 38 | 24    |

```
39 10
40 18
41 31
42 23
```

```
##
ggplot(mapping = aes(y = sim.data.smokers.Poisson$sim_1,
                      x = d.smokers$person)) +
  geom_boxplot() +
  geom_hline(yintercept = 0) +
  ylab("simulated no. of cigarettes\n(assuming Poisson dist)") +
  xlab("person")
```



The results obtained simulating from the Poisson Model are much more similar to the observations that I made up.

## 1.4 Take Home Messages (count data)

- the Linear Model is well suited to model a wide variety of data types
- however, for count data, Linear Models are inappropriate
- for count data, the Poisson Model is more appropriate
- the Poisson Model, assumes a Poisson distribution of the data and uses the natural logarithm as a “link function”

- in **R** the Poisson Model is implemented in the `glm()` function with the argument `family` set to `"poisson"`

## 2 Binary and binomial data

Let's turn our attention to another type of data frequently encountered in the daily work of a data scientist.

### 2.1 Example

#### Binary data (0 or 1)

- whether a client visiting a website clicks on advertisement banner
- whether a cancer patient survived an “invasive” surgery
- whether a machine works properly when tested to do so
- whether a client is able to pay back a loan to his bank

#### Binomial data (number of success / number of trials) $\in [0, 1]$

- the proportion of students that passes an exam (e.g. hopefully all)
- the proportion of trains that gets to destination on time
- the proportion of machines properly working in a batch of 100 pieces (e.g. 96/100)

**Question:** Take your current (or wished) job field and think of an example of binary data and one for binomial data.

### 2.2 The “bliss” data set

Let's use a real binomial data set.

```
library(faraway)
data(bliss)
```

Have a look at the help page of this data set by typing `?bliss`. Let's then look at the structure of the data.

```
str(bliss)
```

```
'data.frame':  5 obs. of  3 variables:
 $ dead : num  2 8 15 23 27
 $ alive: num  28 22 15 7 3
 $ conc : int  0 1 2 3 4
```

```
bliss
```

|   | dead | alive | conc |
|---|------|-------|------|
| 1 | 2    | 28    | 0    |
| 2 | 8    | 22    | 1    |
| 3 | 15   | 15    | 2    |
| 4 | 23   | 7     | 3    |
| 5 | 27   | 3     | 4    |

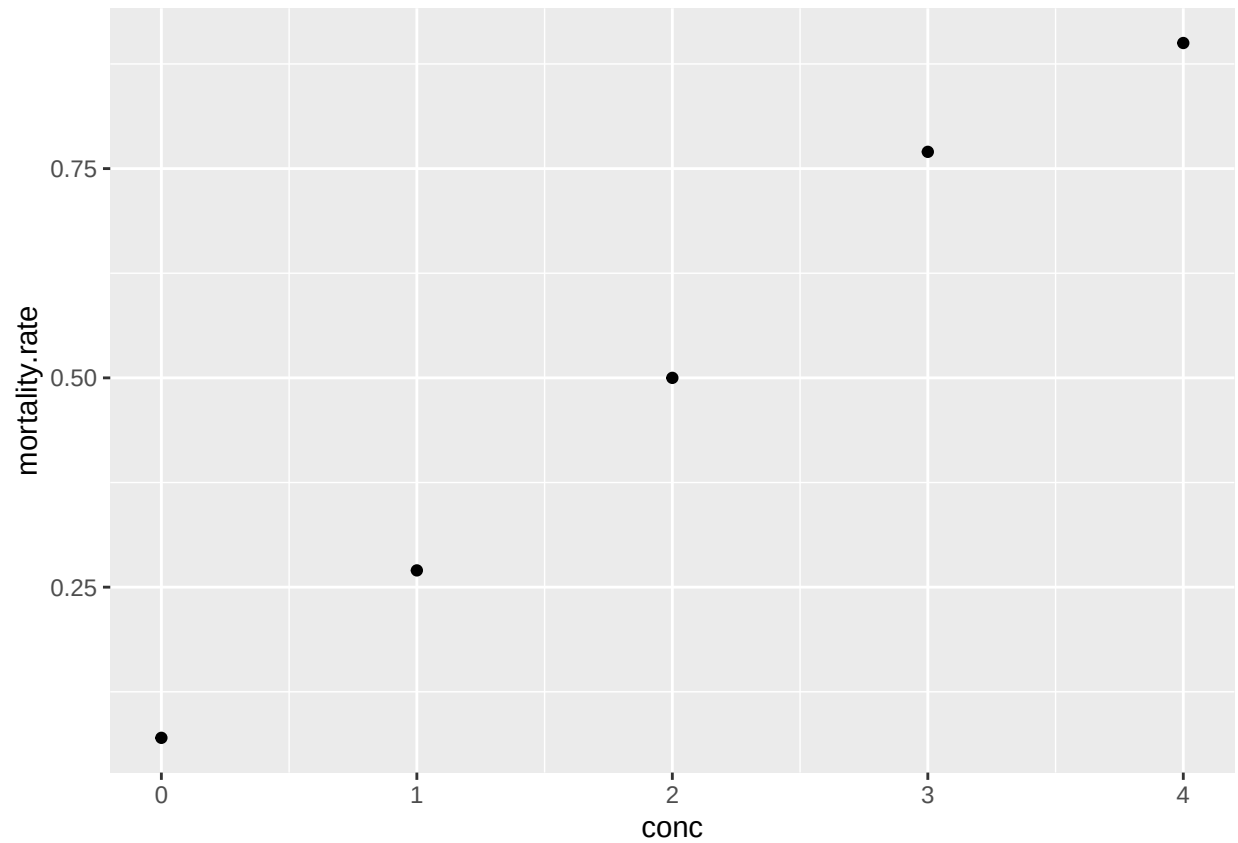
30 insects were exposed to five different insecticide concentrations. After the exposure, it was recorded how many insects survived and how many died (data is aggregated at insecticide concentration level). Let's compute the mortality rate for each concentration.

```
bliss$total.insects <- bliss$dead + bliss$alive
bliss$mortality.rate <- bliss$dead / bliss$total.insects
##
## or with a more "modern" approach
library(dplyr)
bliss <- bliss %>%
  mutate(mortality.rate = round(dead / (dead + alive), digits = 2))
##
bliss
```

|   | dead | alive | conc | total.insects | mortality.rate |
|---|------|-------|------|---------------|----------------|
| 1 | 2    | 28    | 0    | 30            | 0.07           |
| 2 | 8    | 22    | 1    | 30            | 0.27           |
| 3 | 15   | 15    | 2    | 30            | 0.50           |
| 4 | 23   | 7     | 3    | 30            | 0.77           |
| 5 | 27   | 3     | 4    | 30            | 0.90           |

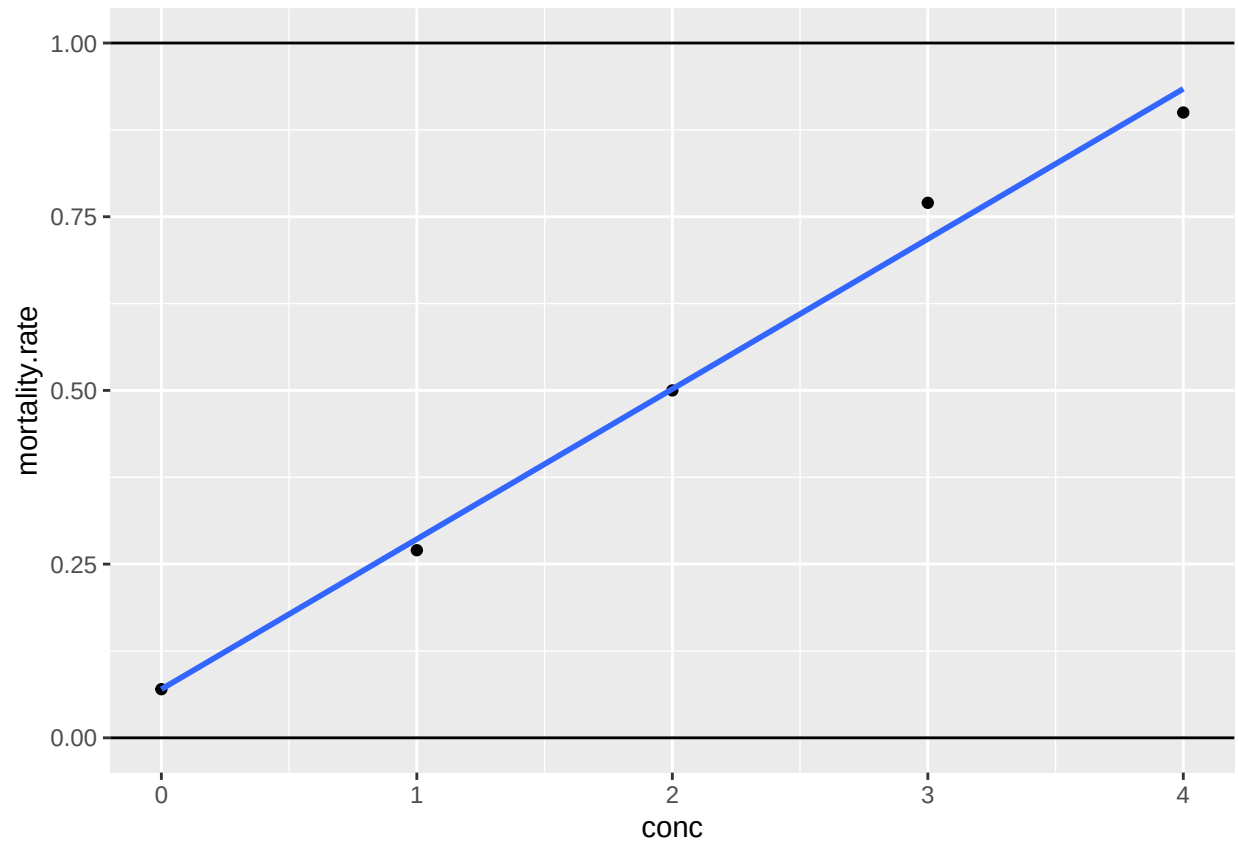
Mortality rates vary from 7% to 90%. Let's display these data.

```
library(ggplot2)
ggplot(data = bliss,
       mapping = aes(y = mortality.rate,
                     x = conc)) +
  geom_point()
```



Note that on this graph the y-axis represents a probability. Therefore, we are adding two horizontal lines, one at 0% and another at 100%. Let's also add a regression line to this graph.

```
ggplot(data = bliss,
       mapping = aes(y = mortality.rate,
                     x = conc)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  ylim(0, 1) +
  geom_hline(yintercept = 0:1)
```



**Question:** *The fit seems to be fairly good. However, what problem could arise when fitting a Linear Model to these data?*

**Question:** *How could we solve this problem?*

**Answers to the question whether a Linear Model model would be appropriate here:**

1. predictions may lay outside the probability range [0%, 100%]
2. even fitted values may lay outside [0%, 100%]
3. actually, the response variable is not normally distributed. Its distribution is bounded between 0% and 100%. The normal distribution is unbounded and thus inconvenient here.
4. the variability of the observations is not constant as it is assumed by the normal distribution. This is not evident on the graph above as there is only one observation for each concentration group. If there were many observations per group, we could see that the variability is not constant along the y-axis. Remember, that the Linear Model assumes that the variability of the observations is fixed and constant.

## 2.3 Link function for binomial data

The solution to problems 1. and 2. is to use a link function such that predictions are constrained in the [0%, 100%] range (note that mathematically, we say that  $y$  must be bounded between 0 and 1).

For binary or binomial data the default choice of the link function is the “logit”.

$$\text{logit}(y) = \log\left(\frac{y}{1-y}\right) \quad y \in [0, 1]$$

So in this case, what is then modelled is:

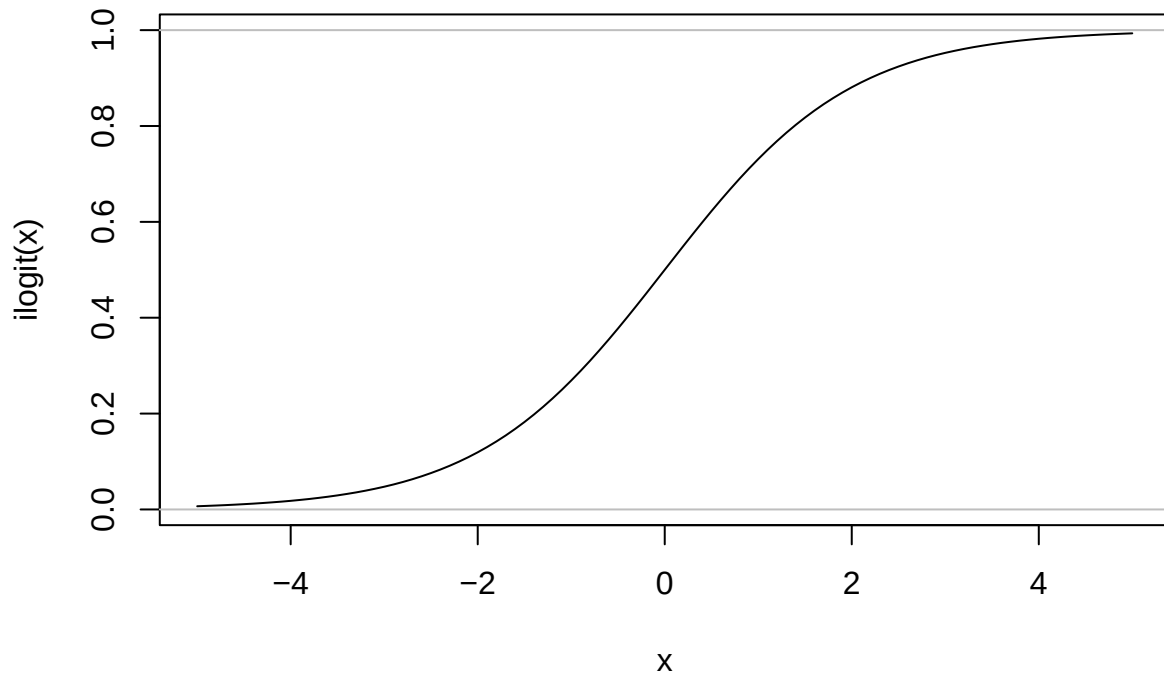
$$\text{logit}(y) = \beta_0 + \beta_1 \cdot x_1$$

Once you applied the link function to the left-hand side of this equation, you can bring it to the right-hand side by taking its inverse on both sides.

$$y = \text{inverse.logit}(\beta_0 + \beta_1 \cdot x_1)$$

The next graph shows the inverse logit function such that you can appreciate how the relationship between response variable and predictor is modelled.

```
curve(expr = ilogit, from = -5, to = 5)
abline(h = c(0, 1), col = "gray")
```



Whatever value is feed to the inverse logit function, the result will be bounded between zero and one. Some examples:

```
ilogit(-20)
```

```
[1] 2.061e-09
```

```
ilogit(-5)
```

```
[1] 0.006693
```

```
ilogit(0)
```

```
[1] 0.5
```

```
ilogit(5)
```

```
[1] 0.9933
```

```
ilogit(20)
```

```
[1] 1
```

As mentioned above a binomial proportion does not follow a normal distribution. Therefore, to solve this problem, the data is assumed to follow a binomial distribution. This way to proceed is in close analogy to what we did to deal with count data.

**In other words, to deal with the peculiarities of count and binomial/binary data, we use a link function and assume a distribution other than the normal.** This is the essence of GLMs.

## 2.4 The Binomial Model in R

Let's fit a Binomial Model to the "bliss" dataset. Note the use of `cbind()` in the formula below. Indeed, the response variable must be specified as `cbind(nr. of successes, nr. of failures)` as it is a binomial proportion. Note that here deaths are considered successes. The reason is that we are testing an insecticide, therefore, when the insecticide is effective the insects are killed<sup>2</sup>.

```
glm.insects <- glm(cbind(dead, alive) ~ conc,
                  family = "binomial",
                  data = bliss)
```

Again, don't forget to set the family when using the `glm()` function.

Let's look at the results of this model.

```
summary(glm.insects)
```

Call:

```
glm(formula = cbind(dead, alive) ~ conc, family = "binomial",
    data = bliss)
```

Coefficients:

|             | Estimate | Std. Error | z value | Pr(> z )    |
|-------------|----------|------------|---------|-------------|
| (Intercept) | -2.324   | 0.418      | -5.56   | 2.7e-08 *** |
| conc        | 1.162    | 0.181      | 6.40    | 1.5e-10 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 64.76327 on 4 degrees of freedom  
Residual deviance: 0.37875 on 3 degrees of freedom  
AIC: 20.85

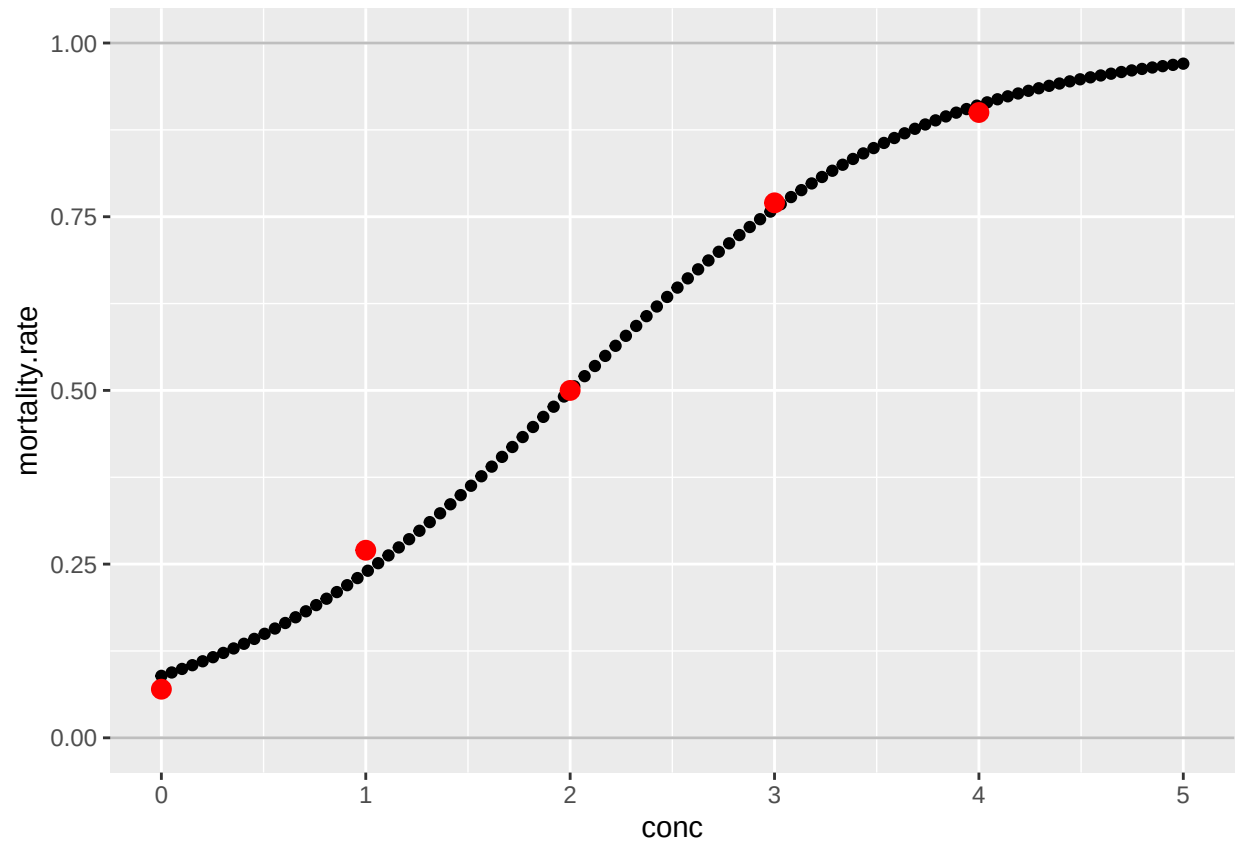
Number of Fisher Scoring iterations: 4

Not unexpectedly concentration seems to play a relevant role. Indeed, its p-value is highly significant. Note that the regression coefficient for *conc* is positive. Indeed, higher concentrations lead to higher number of deaths (i.e. successes). See the Appendix to better understand how the regression coefficients can be interpreted in GLMs.

Let's visualise the results of this model. We plot the actual observed values (red dots) along with predictions for a sequence of 100 concentrations that ranges from zero to five (black dots).

---

<sup>2</sup>Note that you could swap *dead* and *alive* here (i.e. use `cbind(alive, dead)`) and you would get exactly the same results up to the sign of the regression coefficients.



The fit of the model is quite good. Indeed, the predictions are quite close to the real observations. Note the sigmoidal shape of the predicted values. This is due to the use of the link function, which makes sure that fitted or predicted values will always lay within zero and 100%.

## 2.5 Take Home Messages (binary/binomial data)

- binomial and binary data are “mathematically” the same<sup>3</sup>
- binomial or binary data are unsuited to be analysed with Linear Models
- Binomial Models are more appropriate to model these kind of data
- the Binomial Model is implemented in **R** in the `glm()` function when the `family` argument is set to `"binomial"`.

---

<sup>3</sup>Binary data is a special case of binomial data, where the total number of trials is 1.

## 3 Further on binary and binomial data

### 3.1 Binary data: Logistic regression

A few notes on binary data:

- a Binomial Model fitted on binary data (0/1) is often called “logistic regression”
- logistic regression models are fitted via the `glm()` function, by setting the `family` argument to `"binomial"`. Exactly the same way as we do for binomial data.
- When fitting a logistic regression we don't need to use `cbind()`. We can simply provide the response variable coded as 0 and 1 (see example).

```
glm(survived.surgery ~ age + sex,  
    family = "binomial",  
    data = someSurgeryData)
```

### 3.2 Multinomial data (\*)

Up to this point we assumed that there are only two possible outcomes. For example, for the “bliss” dataset we assumed at the end of the experiment an insect is *dead* or *alive*.

Note, however, that in many real applications there are more than two possible outcomes. For example, an insurance client can choose among several possible options. Also in this case the insurer may want to model client choices.

Fortunately, an extension of the logistic regression to more than two classes exist. This method is called “multinomial regression” and is implemented in the `multinom()` function from the `{nnet}` package (see **R** example below).

Example of multinomial regression: The analysis of the “iris” dataset. In this case there are 3 species of flowers (i.e. “setosa”, “versicolor” and “virginica”). The aim of the modelling is to be able to define the species of an iris flower based on its flower measurements (see below).

```
## Iris data  
head(iris)
```

|   | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species |
|---|--------------|-------------|--------------|-------------|---------|
| 1 | 5.1          | 3.5         | 1.4          | 0.2         | setosa  |
| 2 | 4.9          | 3.0         | 1.4          | 0.2         | setosa  |
| 3 | 4.7          | 3.2         | 1.3          | 0.2         | setosa  |
| 4 | 4.6          | 3.1         | 1.5          | 0.2         | setosa  |
| 5 | 5.0          | 3.6         | 1.4          | 0.2         | setosa  |
| 6 | 5.4          | 3.9         | 1.7          | 0.4         | setosa  |

```
table(iris$Species)
```

| setosa | versicolor | virginica |
|--------|------------|-----------|
| 50     | 50         | 50        |

```
##
library(nnet)
multinom.iris <- multinom(Species ~ Sepal.Length + Petal.Width,
                           trace = FALSE,
                           data = iris)
```

### 3.3 Estimating the performance of a binary model

A very common way to estimate the performance of a binary model is to look at predicted values and compare them with the actual observations.

Let's fit a binary model. We use the *stability* data set, which contains two variables:

- *perform*: Indicates whether the employee is able to perform a given task in groups
- *stability*: The score obtained by the employee in a test about "emotional stability"

```
d.stab <- read.table("../Datasets/stability.dat",
                     stringsAsFactors = TRUE,
                     header = TRUE)
str(d.stab)
```

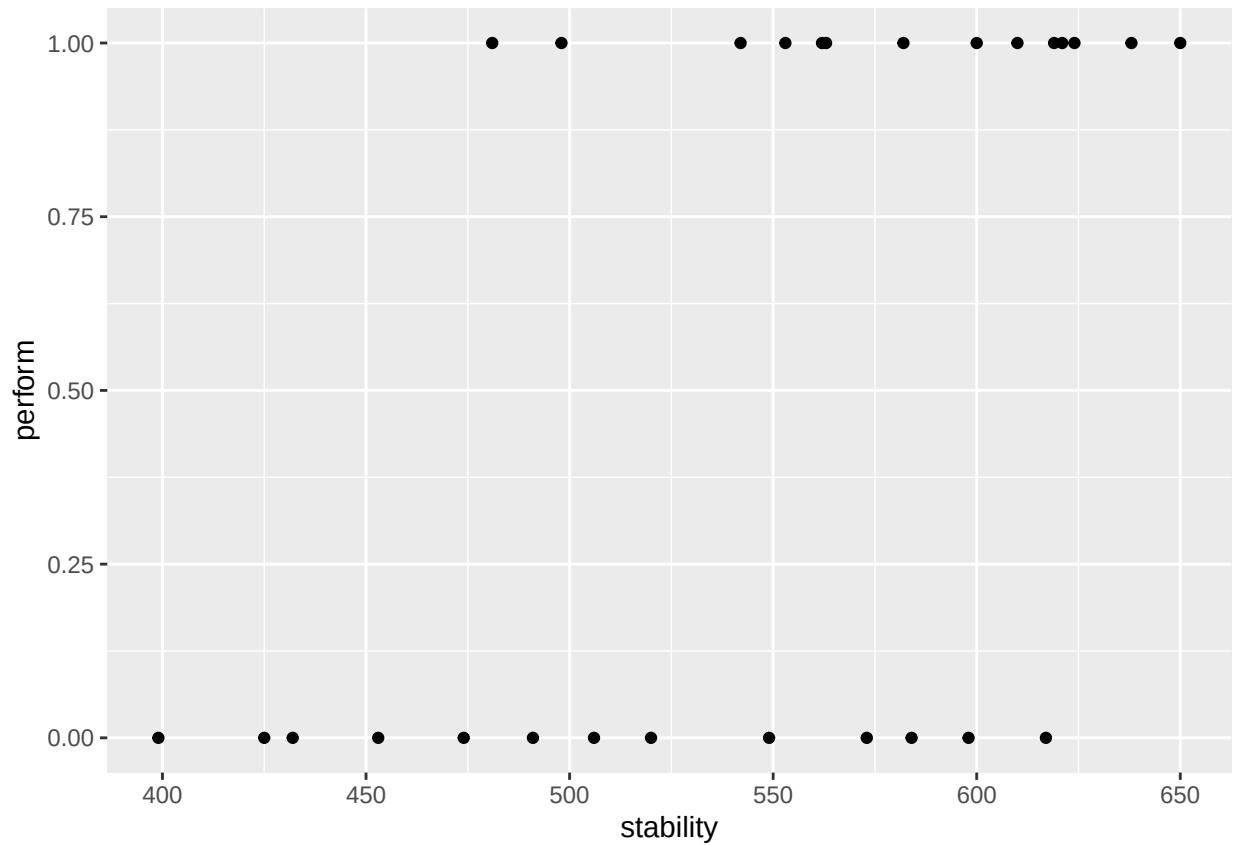
```
'data.frame': 27 obs. of 2 variables:
 $ perform : num 0 0 0 1 1 0 0 1 1 1 ...
 $ stability: num 474 432 453 481 619 584 399 582 638 624 ...
```

```
head(d.stab)
```

|   | perform | stability |
|---|---------|-----------|
| 1 | 0       | 474       |
| 2 | 0       | 432       |
| 3 | 0       | 453       |
| 4 | 1       | 481       |
| 5 | 1       | 619       |
| 6 | 0       | 584       |

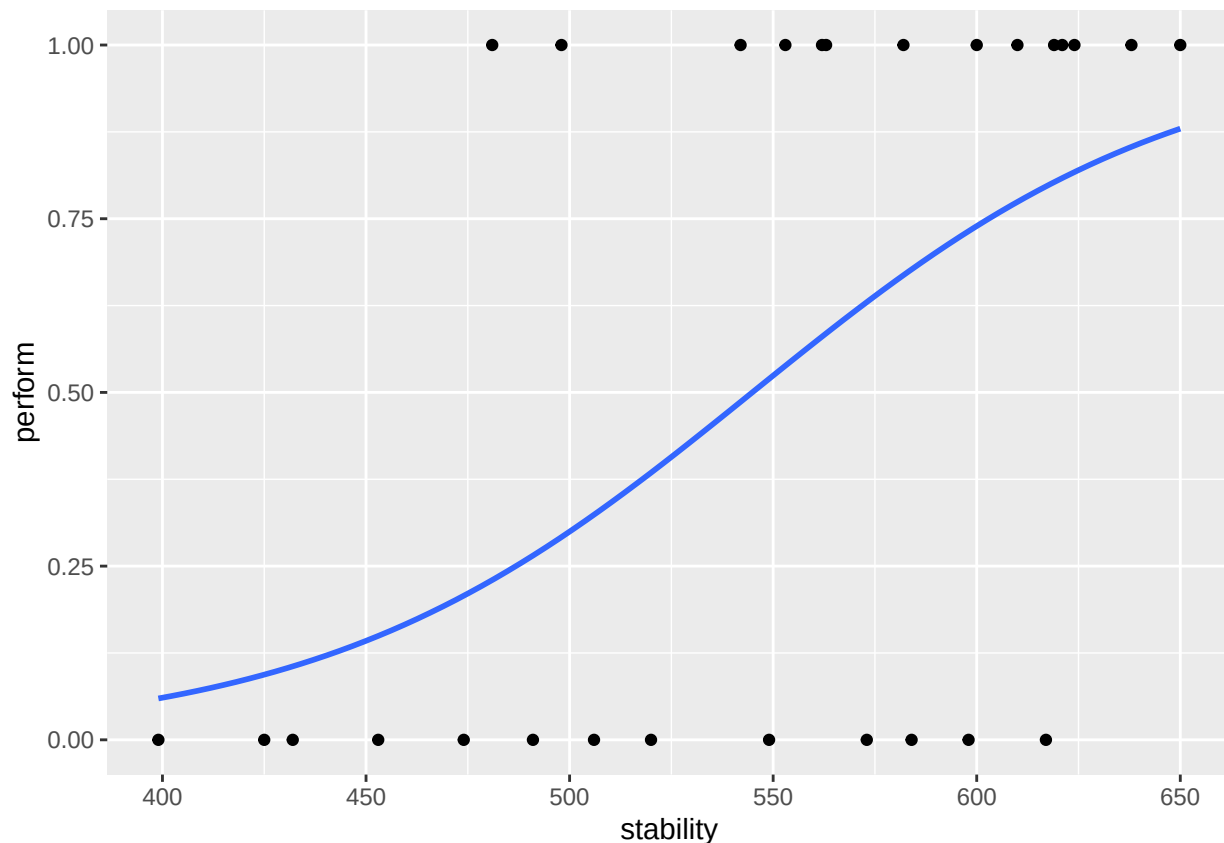
There are 27 employees in this data set. Let's visualise the data.

```
ggplot(data = d.stab,
       mapping = aes(y = perform,
                     x = stability)) +
  geom_point()
```



As you can see there are only two possible outcomes here: success or failure (they are coded as one and zero respectively). Successes seem to be associated with higher values of *stability*. This makes sense, as more emotionally stable employees are expected to be more likely to successfully perform tasks in groups.

Let's fit a logistic regression model and add the fit to the this graph (code not shown).



As expected higher stability scores are associated with higher probabilities to successfully perform group tasks.

Let's fit a logistic regression to these data.

```
glm.stab <- glm(perform ~ stability,
  data = d.stab,
  family = "binomial")
summary(glm.stab)
```

Call:

```
glm(formula = perform ~ stability, family = "binomial", data = d.stab)
```

Coefficients:

|             | Estimate  | Std. Error | z value | Pr(> z ) |
|-------------|-----------|------------|---------|----------|
| (Intercept) | -10.30893 | 4.37700    | -2.36   | 0.019 *  |
| stability   | 0.01892   | 0.00788    | 2.40    | 0.016 *  |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 37.393 on 26 degrees of freedom  
Residual deviance: 29.242 on 25 degrees of freedom

AIC: 33.24

Number of Fisher Scoring iterations: 4

As expected *stability* has a positive effect on the response variable. Let's have a look at the fitted values for this model. Note that we round all fitted values to two digits.

```
fitted(glm.stab) %>% round(digits = 2)
```

| 1    | 2    | 3    | 4    | 5    | 6    | 7    | 8    | 9    | 10   | 11   | 12   | 13   | 14   | 15   | 16   |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 0.21 | 0.11 | 0.15 | 0.23 | 0.80 | 0.68 | 0.06 | 0.67 | 0.85 | 0.82 | 0.49 | 0.88 | 0.54 | 0.09 | 0.58 | 0.52 |
| 17   | 18   | 19   | 20   | 21   | 22   | 23   | 24   | 25   | 26   | 27   |      |      |      |      |      |
| 0.29 | 0.38 | 0.77 | 0.73 | 0.27 | 0.80 | 0.81 | 0.63 | 0.58 | 0.32 | 0.74 |      |      |      |      |      |

Note that they are probabilities, thus, bounded within zero and one. In other words, none of the employees will get a fitted values that is smaller than zero or larger than one.

To compare the fitted values with the observed values, which are indeed binary (0 or 1), we can also discretise/dichotomise the fitted values into 0 and 1. In order to do that, we use a cutoff of 0.5.

```
fitted.stab.disc <- ifelse(fitted(glm.stab) < 0.5,  
                           yes = 0, no = 1)  
head(fitted.stab.disc)
```

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 1 |

The value used to discretise/dichotomise probabilities is called **cutoff**. In virtually all settings its default value is 0.5.

Finally, let's compare the observed and fitted values.

```
d.obs.fit.stab <- data.frame(obs = d.stab$perform,  
                             fitted = fitted.stab.disc)  
head(d.obs.fit.stab)
```

|   | obs | fitted |
|---|-----|--------|
| 1 | 0   | 0      |
| 2 | 0   | 0      |
| 3 | 0   | 0      |
| 4 | 1   | 0      |
| 5 | 1   | 1      |
| 6 | 0   | 1      |

The best way to compare the observed and the discretised/dichotomised fitted values is to summarise them into a table.

```
table(d.obs.fit.stab$obs)
```

| 0  | 1  |
|----|----|
| 13 | 14 |

```
## 14 success and 13 failures in the real data
##
table(obs = d.obs.fit.stab$obs,
      fit = d.obs.fit.stab$fitted)
```

```
      fit
obs  0  1
  0  8  5
  1  3 11
```

The diagonal entries of this matrix represent correctly labelled observations. For example, 8 observations were correctly labelled to be failures and the same time 11 observations were correctly labelled to be successes. However, 5 observations were wrongly classified (by this model) to be successes when in reality they are failures. On the other hand, the model misclassified 3 observations as being failures, when in reality they were not. This table is referred to as the “confusion matrix”<sup>4</sup>.

Note that the confusion matrix can also be expressed as percentages, rather than counts in each cell.

```
table(obs = d.obs.fit.stab$obs,
      fit = d.obs.fit.stab$fitted) %>%
  prop.table() %>%
  round(digits = 2)
```

```
      fit
obs  0    1
  0 0.30 0.19
  1 0.11 0.41
```

So, we can state that 71% of the observations were correctly labelled (30% + 41% i.e. the two diagonal entries). While, for example 19% of the observations were wrongly labelled as successes (i.e. “1”), when in reality they were “0”.

Note that there is much more to confusion matrices to evaluate the performance of a binary model. For further information see AUC and ROC analysis in the reference book (or the **R**-packages *{pROC}* and *{ROCR}*).

## 4 Generalised Linear Models

Both the Poisson and the Binomial Model are members of a group of models called **Generalised Linear Models** (GLMs for short). They are an extension of the Linear Model to deal with particular types of data.

In particular, these models:

- assume a distribution of the data (here we have seen Binomial and Poisson, but other exist)
- use a link function
- can be used for testing hypotheses and estimating effects

---

<sup>4</sup>I guess because... it always confuses students ;)

## 5 Appendix

The appendix is there for the sake of completeness. Note, however, that this does not exclude that in the exam there may be a few questions relating to topics only seen in the appendix. Nevertheless, keep in mind that topics treated in the appendix will constitute at most 5% of the exam.

### 5.1 Interpreting the coefficients of a Poisson Model (\*)

The *esdcomp* dataset in package *{faraway}* records the number of complaints received by 44 doctors working in an emergency service at a hospital (type `?esdcomp` for more information). let's fit an appropriate model to these data.

```
glm.complaints <- glm(complaints ~ . , data = esdcomp,
                      family = "poisson")
```

We can then look at the summary output.

```
summary(glm.complaints)
```

Call:

```
glm(formula = complaints ~ . , family = "poisson", data = esdcomp)
```

Coefficients:

|             | Estimate  | Std. Error | z value | Pr(> z ) |
|-------------|-----------|------------|---------|----------|
| (Intercept) | -0.080345 | 1.154212   | -0.07   | 0.945    |
| visits      | 0.000950  | 0.000339   | 2.81    | 0.005 ** |
| residencyY  | -0.231974 | 0.202939   | -1.14   | 0.253    |
| sexM        | 0.112239  | 0.223504   | 0.50    | 0.616    |
| revenue     | -0.003383 | 0.004155   | -0.81   | 0.416    |
| hours       | -0.000157 | 0.000663   | -0.24   | 0.813    |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 89.447 on 43 degrees of freedom  
Residual deviance: 49.995 on 38 degrees of freedom  
AIC: 184.8

Number of Fisher Scoring iterations: 5

The structure of the summary output is similar to the one obtain when using the `lm()` function. All the differences in the summary structure are due to the fact that we assume a Poisson distribution of the data (e.g. **Deviance residuals** instead of **residuals** are reported at the top of the summary).

#### 5.1.1 Interpreting the coefficients of a Poisson Model: Factors (\*)

Let's turn our attention at the estimated coefficients. The interpretation of the model coefficients differs from a plain Linear Model. For example, let's look at the effect of *sex*.

```
coef(glm.complaints)["sexM"]
```

```
sexM  
0.1122
```

To properly interpret this coefficient, we must bare in mind that the link function used here is the `log()` function (i.e. the natural logarithm). Therefore, the correct interpretation of this model coefficient is obtained by taking its inverse, the exponential:

```
exp(coef(glm.complaints)["sexM"]) %>% round(digits = 2)
```

```
sexM  
1.12
```

The correct interpretation of this coefficient is: “Male doctors get, on average, 12% more complaints than women doctors.” Note that the interpretation is not additive as in the Linear Model (i.e. we don’t say that male doctors get on average 1.12 more complaints for males), but rather multiplicative (i.e. 12% more complaints for males). Again, this difference in interpretation is due to the use of the logarithm as a link function.

To convince ourselves of the multiplicative effect of the predictors, we can look at single doctor and its predicted value. We then modify the sex of this doctor and look at the predicted value again.

Let’s say we take the very first doctor:

```
esdcomp[1, ]
```

```
visits complaints residency sex revenue hours  
1 2014 2 Y F 263 1287
```

In this case, the first doctor is a women. Let’s look at predicted number of complaints for this doctor:

```
fitted.first.doctor <- fitted(glm.complaints)[1]  
fitted.first.doctor
```

```
1  
1.664
```

This doctor, according to the model, is expected to have received 1.66 complaints. Let’s see what the expected number of complaints for the same doctor, but with sex set to male:

```
first.doctor.as.male <- esdcomp[1, ]  
first.doctor.as.male$sex <- "M"  
first.doctor.as.male
```

```
visits complaints residency sex revenue hours  
1 2014 2 Y M 263 1287
```

```
##
pred.first.doc.male <- predict(glm.complaints,
  type = "response", # ! important to set argument "type" to response!
  newdata = first.doctor.as.male)
pred.first.doc.male
```

```
1
1.861
```

For the first doctor, when *sex* is set to male, the model predicts the expected number of complaints to be 1.86. Let's now see whether this corresponds to the expected number of complaints for the first doctor times the exponential estimated for sex male (i.e. 1.12).

```
fitted.first.doctor * exp(coef(glm.complaints)["sexM"])
```

```
1
1.861
```

Indeed, the first doctor with sex set to male would receive, on average,  $1.66 * 1.12 = 1.86$  complaints.

Another example would be two women doctors that have a predicted number of complaints of 10 and respectively 50. According to the model, if these two doctors were males, then we would expect them to have received 11.2 and respectively 56 complaints.

### 5.1.2 Interpreting the coefficients of a Poisson Model: Continuous predictors (\*)

Let's turn our attention to the continuous predictor *visits*. As expected, this coefficient is positive, as the more patients a given doctor visited, the higher the number of complaints he/she gets.

```
exp.coef.visits <- exp(coef(glm.complaints)["visits"])
print(exp.coef.visits, digits = 5)
```

```
visits
1.001
```

The correct interpretation for this coefficient is: "For a given doctor, increasing its number of visits by one, would result in about 0.1% more complaints."

It is clear that to get a more realistic interpretation of this coefficient, we may want to increase the number of visits by more than one unit.

To select a more meaningful increase in visits, we first look at the range of values found in *visits*.

```
range(esdcomp$visits)
```

```
[1] 879 3763
```

The doctor that made fewer visits, visited almost 900 patients. Let's try to interpret our model coefficient when the number of visits is increased by 50.

```
coef.visits.50 <- coef(glm.complaints)["visits"] * 50
coef.visits.50
```

```
visits
0.0475
```

```
##
print(exp(coef.visits.50), digits = 5)
```

```
visits
1.0486
```

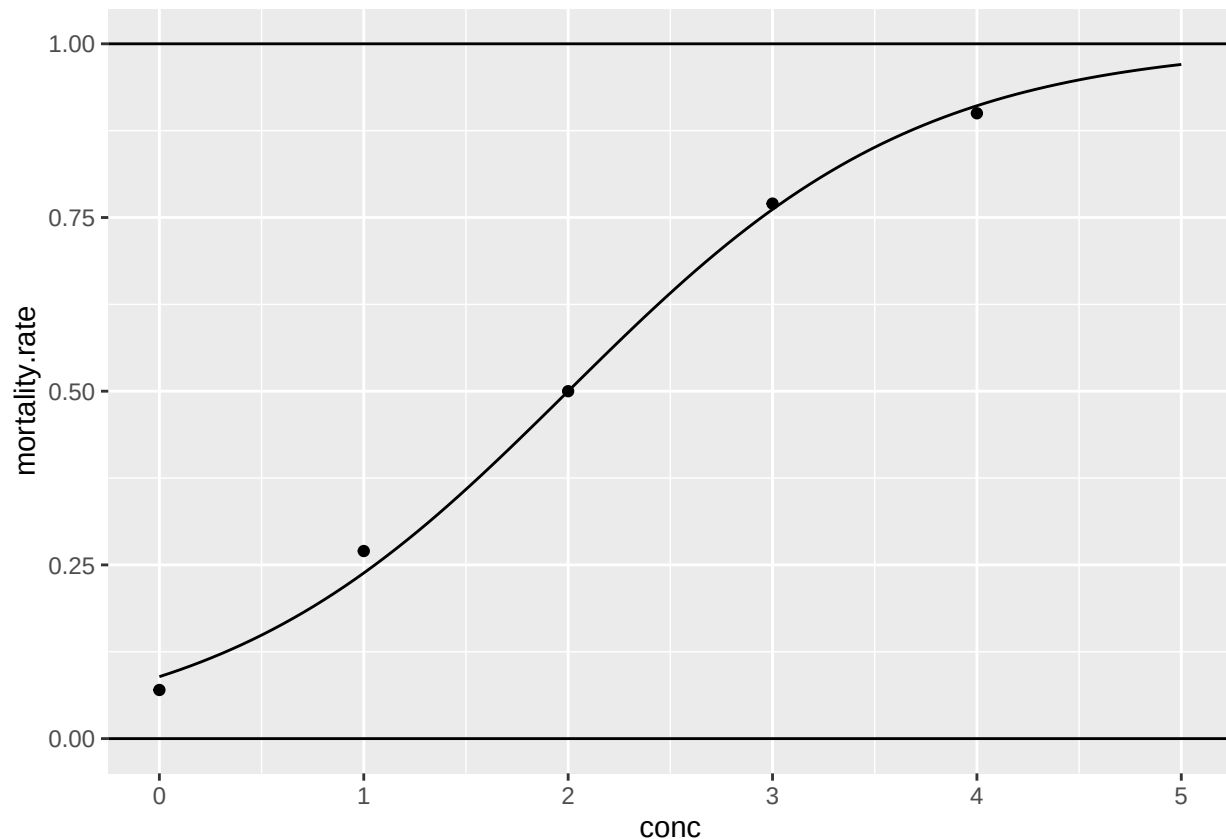
For a given doctor, if you were to increase its number of visits by 50, then we expect this doctor to get about 4.86% more complaints<sup>5</sup>.

## 5.2 Interpreting the coefficients of a Binomial Model (\*\*)

Let's get back to the analysis of the “bliss” dataset. Let's visualise the data along with the model fit (note the code is not shown, but accessible in the corresponding “R” file).

### 5.2.1 Interpreting the coefficients of a Binomial Model: Continuous predictors (\*\*)

Before looking at the estimated regression we display the model fit along with the data.



---

<sup>5</sup>Note that multiplying coefficients by a given factor to get more realistic interpretation of the estimated effects is not a peculiarity of the Poisson Model. This can be done with any Linear Model or GLM.

Note that this sigmoid fit is symmetric around the 50%. The steepness of this fit is proportional to the estimated coefficient. A large regression coefficient for *conc* would imply a steeper raise in mortality.

Let's extract the model coefficients from the fitted object.

```
coef(glm.insects)
```

```
(Intercept)      conc  
      -2.324      1.162
```

As link functions are used the interpretation of these coefficients must be adapted. In particular, we can interpret the *conc* coefficient by applying the exponential function.

```
exp(coef(glm.insects)["conc"])
```

```
conc  
3.196
```

By increasing the concentration of the insecticide by one unit, we will obtain an increased risk in the odds<sup>6</sup> of about 3 times.

To better understand the sentence above, we can add the fitted values for two values of *conc*: 2 and 3.

```
Warning in geom_segment(mapping = aes(x = 2, y = 0, xend = 2, yend = 0.5), : All aesthetics have length  
1 Please consider using 'annotate()' or provide this layer with data containing  
a single row.
```

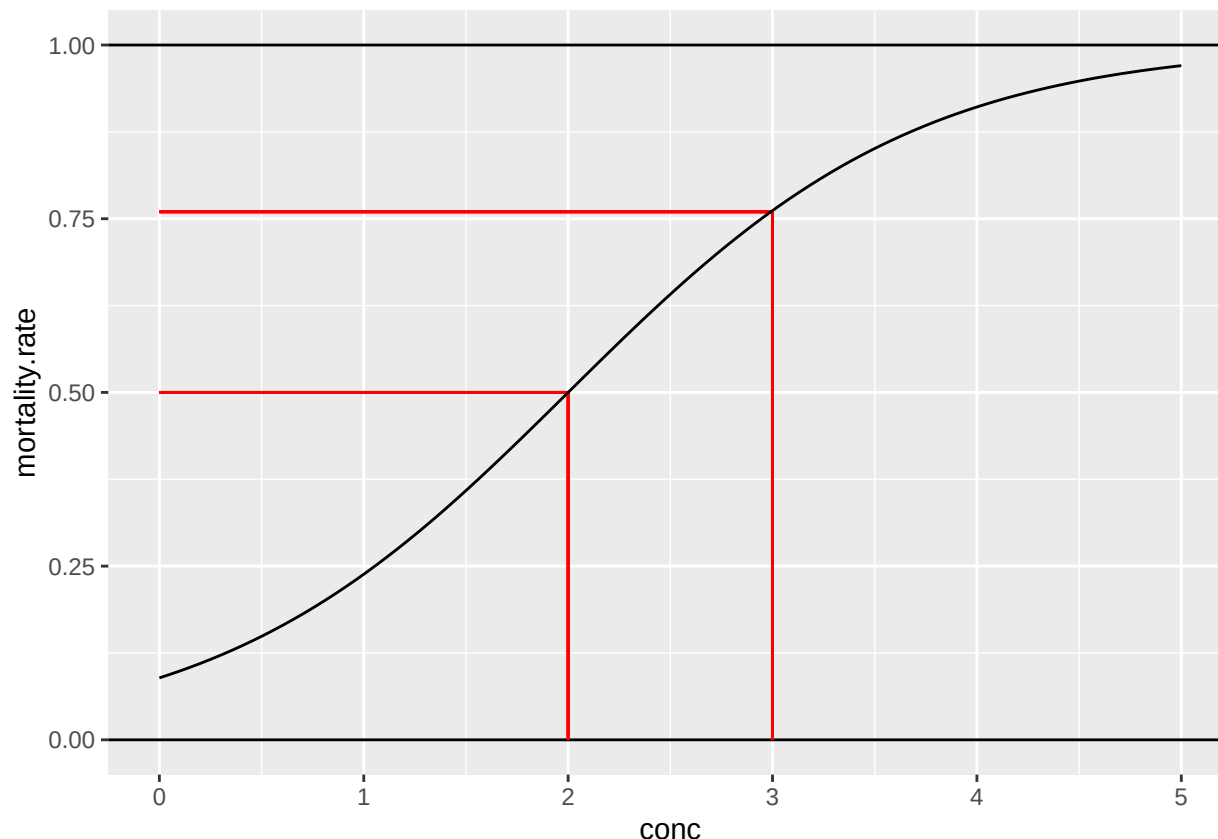
```
Warning in geom_segment(mapping = aes(x = 0, y = 0.5, xend = 2, yend = 0.5), : All aesthetics have leng  
1 Please consider using 'annotate()' or provide this layer with data containing  
a single row.
```

```
Warning in geom_segment(mapping = aes(x = 3, y = 0, xend = 3, yend = 0.76), : All aesthetics have lengt  
1 Please consider using 'annotate()' or provide this layer with data containing  
a single row.
```

```
Warning in geom_segment(mapping = aes(x = 0, y = 0.76, xend = 3, yend = 0.76), : All aesthetics have le  
1 Please consider using 'annotate()' or provide this layer with data containing  
a single row.
```

---

<sup>6</sup>odds refer to  $\frac{p}{1-p}$ , where p is the probability of success.



The expected mortality (i.e.  $p$ ) for a insecticide concentration of 2 is about 50%. Therefore, the odds to survive at concentration 2 are  $\frac{p}{1-p} = \frac{0.5}{1-0.5} = 1 : 1$ . In other words, with a concentration of 2 insects are as likely to die as to survive.

Let's now move to a higher concentration:  $conc = 3$ . In this case, the probability to die is about 75%. Therefore, the odds are  $\frac{0.75}{1-0.75} = \frac{0.75}{0.25} = 1 : 3$ . In other words, with a insecticide concentration of 3, insects are three times more likely to die than to survive.

This brings us back to interpretation of the coefficient of *conc*:

```
exp(coef(glm.insects)["conc"])
```

```
conc
3.196
```

By increasing the concentration of the predictor *conc* by one unit, the odds of dying have a three-fold increase. **Indeed, moving from concentration 2 to 3, we observed a threefold increase in the odds of dying (i.e. from 1:1 to 1:3).**

### 5.2.2 Interpreting the coefficients of a Binomial Model: Factors (\*\*)

In this very simple dataset there are no factors. Therefore, we discretise *conc* such that it becomes a factor with 3 levels.

```
bliss$conc.asFactor <- cut(bliss$conc, breaks = c(-Inf, 0, 2, 4),
                           labels = c("no insecticide", "low conc", "high conc"))
levels(bliss$conc.asFactor)
```

```
[1] "no insecticide" "low conc"      "high conc"
```

Let's double check that the discretisation worked well.

```
with(bliss, table(conc, conc.asFactor))
```

|      | conc | no insecticide | low conc | high conc |
|------|------|----------------|----------|-----------|
| conc | 0    | 1              | 0        | 0         |
|      | 1    | 0              | 1        | 0         |
|      | 2    | 0              | 1        | 0         |
|      | 3    | 0              | 0        | 1         |
|      | 4    | 0              | 0        | 1         |

Let's fit the corresponding Poisson Model.

```
glm.insects.factor <- glm(cbind(dead, alive) ~ conc.asFactor,
                          data = bliss,
                          family = "binomial")
```

Let's get the coefficients.

```
coef(glm.insects.factor)
```

|  | (Intercept) | conc.asFactorlow conc | conc.asFactorhigh conc |
|--|-------------|-----------------------|------------------------|
|  | -2.639      | 2.164                 | 4.248                  |

Let's interpret them:

```
exp(coef(glm.insects.factor))
```

|  | (Intercept) | conc.asFactorlow conc | conc.asFactorhigh conc |
|--|-------------|-----------------------|------------------------|
|  | 0.07143     | 8.70270               | 70.00000               |

The odds of dying for a low insecticide concentration are about 8.7 times higher than the odds for the “no insecticide” settings.

### 5.2.3 The “divide by 4 rule”

The logistic curve is steepest at its center, i.e. when  $x$  is equal to its mean  $\bar{x}$ , at which point  $\alpha + \beta x = 0$  (indeed  $y = 0.5 = \text{logit}^{-1}(0)$ , meaning that  $\alpha + \beta x = 0$ ).

The slope of the curve (i.e. the derivative of the logistic function) is maximised at this point.

Let's calculate it. The formula is

$$\text{logit}(y) = \log\left(\frac{y}{1-y}\right) = \alpha + \beta x,$$

thus if we want to find the value of  $y$ , we have to inverse the logit function:

$$y = \frac{e^{\alpha + \beta x}}{1 + e^{\alpha + \beta x}}.$$

Consequently, the derivative with respect to  $x$  is:

$$y' = \frac{\beta e^{\alpha + \beta x}}{(1 + e^{\alpha + \beta x})^2}.$$

At  $y = 0.5$ , i.e., when  $\alpha + \beta x = 0$ , we have

$$y = \frac{\beta e^0}{(1 + e^0)^2} = \frac{\beta}{4}.$$

Thus,  $\frac{\beta}{4}$  is the maximum difference in the probability of having  $y = 1$ , corresponding to a unit difference in  $x$ .

As a rule of convenience, we can take logistic regression coefficients and divide them by 4 to get an upper bound of the predictive difference corresponding to a unit change in  $x$ . This upper bound is a reasonable approximation near the midpoint of the logistic curve, where probabilities are close to 0.5.

Note that the same reasoning is also valid for factor variables.

Let's use again the *bliss* data set to see a concrete example.

```
glm.insects <- glm(cbind(dead, alive) ~ conc,
                  family = "binomial",
                  data = bliss)
summary(glm.insects)
```

Call:

```
glm(formula = cbind(dead, alive) ~ conc, family = "binomial",
    data = bliss)
```

Coefficients:

|             | Estimate | Std. Error | z value | Pr(> z )    |
|-------------|----------|------------|---------|-------------|
| (Intercept) | -2.324   | 0.418      | -5.56   | 2.7e-08 *** |
| conc        | 1.162    | 0.181      | 6.40    | 1.5e-10 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 64.76327 on 4 degrees of freedom  
 Residual deviance: 0.37875 on 3 degrees of freedom  
 AIC: 20.85

Number of Fisher Scoring iterations: 4

For example, in the model  $\text{Pr}(\text{alive}) = \text{logit}^{-1}(-2.32 + 1.16 \cdot \text{conc})$ , we can divide 1.16 by 4 and we get 0.29: a difference of 1 in the insecticide concentration corresponds to no more than a 29% positive difference in the probability of an insect being alive.

## 5.3 Overdispersion (\*\*)

### 5.3.1 Overdispersion: In Poisson Models (\*\*)

You may have noticed that below the coefficients table in the summary output of the Poisson Models the following sentence is printed (*Dispersion parameter for poisson family taken to be 1*). This highlights that by fitting a Poisson Model we assume that the variance increases linearly with the mean.

In other words, a doctor expected to have received 10 complaints will have a way larger variance than a doctor expected to have received two complaints. To better understand this, you can imagine to repeat this study a large number of times. When looking at the observations of the doctor with two expected complaints, you will observe a variance of two. On the other hand, the variance for the doctor with 10 expected complaints will be 10 (i.e. for Poisson Models the variance equals the mean value).

**In real settings, however, it is very often the case that count data do NOT follow a Poisson distribution.** Most often this type of data shows a variability that is larger than expected under the Poisson distribution. Therefore, in most real settings count data is found to be “**overdispersed**”.

To check whether the data are overdispersed, we can compare the *Residual deviance* and the corresponding *degrees of freedom* in the summary output. Here, the residual deviance is 49.995 on 38 degrees of freedom. If the data was truly Poisson distributed, we would have expected the residual deviance and the degrees of freedom to be (approximately) the same value. There is thus an indication that the data is overdispersed.

**Remember that overdispersion is the rule, not the exception when dealing with count data.**

Is it possible to allow the model to estimate overdispersion via an additional parameter (i.e. the dispersion parameter). We do this by setting the family to “quasipoisson”.

```
quasi.glm.complaints <- glm(complaints ~ . ,
                             data = esdcomp,
                             family = "quasipoisson")
```

We can then look at the summary output.

```
summary(quasi.glm.complaints)
```

Call:

```
glm(formula = complaints ~ ., family = "quasipoisson", data = esdcomp)
```

Coefficients:

|             | Estimate  | Std. Error | t value | Pr(> t ) |
|-------------|-----------|------------|---------|----------|
| (Intercept) | -0.080345 | 1.328486   | -0.06   | 0.95     |
| visits      | 0.000950  | 0.000390   | 2.44    | 0.02 *   |
| residencyY  | -0.231974 | 0.233580   | -0.99   | 0.33     |
| sexM        | 0.112239  | 0.257251   | 0.44    | 0.67     |
| revenue     | -0.003383 | 0.004783   | -0.71   | 0.48     |
| hours       | -0.000157 | 0.000764   | -0.21   | 0.84     |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for quasipoisson family taken to be 1.325)

```
Null deviance: 89.447  on 43  degrees of freedom
Residual deviance: 49.995  on 38  degrees of freedom
AIC: NA
```

Number of Fisher Scoring iterations: 5

Note that the dispersion parameter is now estimated to be 1.32. Before, in the Poisson Model it was fixed to the value 1. This implies that the variance increases faster than linearly.

It is important to note that the estimated model coefficients are identical to those in the Poisson Model. What really changes here are the standard errors of the estimated regression coefficient and as a consequence the associated p-values. In this case, the conclusions would not change dramatically: There is some evidence that the number of visits has an influence on the response variable, while there is little to no evidence that the other variables have an effect.

### 5.3.2 Overdispersion: In Binomial Models (\*\*)

Analogously, also binomial data is often found to be overdispersed in real settings. To cope with that we can use the “quasibinomial” family.

Note that binary data cannot show overdispersion. In other words, you can only fit a model with family set to “binomial” when dealing with binary data.

## 5.4 GAMs and GLMs

Last week we dealt with Generalised Additive Model. This week with GLMs. The question that naturally raises is “Can we mix them? Can fit a GAM on count or binomial data?”

The answer is “Yes, you can”. Indeed, the `gam()` function also has a `family` argument. This argument can be set to “poisson”, “quasipoisson”, “binomial” or “quasibinomial”. Below we show an example with the *esdcomp* dataset.

```
library(mgcv)
gam.complaints <- gam(complaints ~ sex +
                      s(visits) +
                      s(revenue),
                      family = "quasipoisson",
                      data = esdcomp)
##
summary(gam.complaints)
```

Family: quasipoisson  
Link function: log

Formula:  
complaints ~ sex + s(visits) + s(revenue)

Parametric coefficients:  
                    Estimate Std. Error t value Pr(>|t|)  
(Intercept)      0.926      0.205      4.52 5.5e-05 \*\*\*

```
sexM          0.191      0.231      0.83      0.41
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
            edf Ref.df      F p-value
s(visits)  1.00   1.00 25.0 1.2e-05 ***
s(revenue) 1.43   1.74  1.8   0.26
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.448   Deviance explained = 44.1%
GCV = 1.4037   Scale est. = 1.2227      n = 44
```

## 5.5 What if my predictors are e.g. count data?

All what was discussed so far refer to the response variable. What if for example one of my predictors is also count data.

Let's say I want to model survival after surgery and one the predictors is how many body-organs where touched by the surgery. In this case the response is binary the predictors is count data? What methods shall I use?

We don't care about predictors when deciding what method to use to analyse data.

⇒

The only thing that matters in deciding which method is to be used is the response variable. In this example example the response variable is binary and therefore we will fit a GLM with family set to "binomial".

The reasoning applies in the opposite situation where the response is count data and the predictors is a binary factor. There, we will work with a GLM with family set to "quasipoisson".

## 5.6 Not all proportions are binomial data

Binomial data always refer to a ratio among two integers. For example, 47 pieces of meat out of 100 tested contained a given bacteria. In practice we also encounter proportions between **continuous variables** that are not to be treated as binomial data.

Let's say you run an experiment where you provide an animal with 1kg of food an then record how much food was actually consumed. In this case the response variable "proportion of food consumed" is not binomial data and using a binomial GLM would be wrong. On the other hand, if you were to feed you animals with say 10 insects and then recorded how many were eaten you can treat that as binomial data and analyse it with a binomial GLM.

You can see binomial data a something that was tested several times and has only two possible outcomes. For the insects as food you may say that you tried 10 times and you want to see how many times the animal ate the insect. On the other hand 1kg can also be expressed as 1'000g or 0.001 tons.

The point is that in true binomial data the number of "trials" is essential to define the precision of each observation. Say you want to test how many frozen meat pieces contain bacteria. Say you run the experiment twice and get for both cases the result that 50% of meat pieces were contaminated. Apparently the contamination is the same. But let's say that the 50% estimate is based on two pieces of meat (one contaminated) in one case and on 100 pieces of meat (50 samples contaminated) in the other case. Obviously, you trust much more the 50% estimate in the second much larger sample. So, for binomial data the number of trial is defining the precision of the observation itself.

Now with a proportion between two continuous variables there is no “fundamental unit”. What is the unit for the food example? Is it 1g or 10g? So, this is not binomial data and should not be analysed with a GLM binomial.

A proportion between two continuous variables can be analysed within the LM framework. Very often the appropriate transformation of the response variable is needed to make sure that the model assumptions about the errors are fulfilled<sup>7</sup>.

Note that scores are also not to be considered binomial data. If you analyse, say a score that goes from zero to 10, this is not binomial data.

---

<sup>7</sup>Proportions should be transformed with the arc-sine-square-root transformation when analysed within the Linear Models framework. In **R** `lm(asin(sqrt(percentage)) ~ predictors)`.

## Session Information

```
sessionInfo()
```

```
R version 4.5.1 (2025-06-13)
```

```
Platform: aarch64-apple-darwin20
```

```
Running under: macOS Sequoia 15.6.1
```

```
Matrix products: default
```

```
BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
```

```
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK vers
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: Europe/Zurich
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] mgcv_1.9-3    nlme_3.1-168  nnet_7.3-20   dplyr_1.1.4   faraway_1.0.9
```

```
[6] ggplot2_3.5.2 knitr_1.50
```

```
loaded via a namespace (and not attached):
```

```
[1] Matrix_1.7-3      gtable_0.3.6      compiler_4.5.1     Rcpp_1.0.14
[5] tidyselect_1.2.1  tinytex_0.57      splines_4.5.1      scales_1.4.0
[9] boot_1.3-31       yaml_2.3.10       fastmap_1.2.0      lattice_0.22-7
[13] R6_2.6.1          labeling_0.4.3     generics_0.1.4     rbibutils_2.3
[17] MASS_7.3-65       tibble_3.3.0      nloptr_2.2.1       minqa_1.2.8
[21] pillar_1.10.2     RColorBrewer_1.1-3 rlang_1.1.6        xfun_0.52
[25] cli_3.6.5         withr_3.0.2       magrittr_2.0.3     Rdpack_2.6.4
[29] digest_0.6.37     grid_4.5.1        rstudioapi_0.17.1  lme4_1.1-37
[33] lifecycle_1.0.4   reformulas_0.4.1  vctrs_0.6.5        evaluate_1.0.4
[37] glue_1.8.0        farver_2.1.2      rmarkdown_2.29     tools_4.5.1
[41] pkgconfig_2.0.3   htmltools_0.5.8.1
```